OBSCURE // FOUNDER MASTERY

ARCHITECTURE • DEFENSE • REVERSE-ENGINEERING

Obscure Founder Mastery Engineering Handbook

A concise, conceptual guide to Chromium internals, the Shim Pattern, Git safety, and technical defense.

CURRICULUM TRACK	CHROMIUM BASELINE	ARCHITECTURE PATTERN	MASTERY GOAL
Founder & Executive Lead	**Stable 155.0.8059.39**	**The Shim Pattern (≤10 Lines)**	**100% Defensible Knowledge**

01 Chromium Multi-Process Architecture: The "OS Inside an OS"

The Mental Model: Why Monolithic Browsers Failed

In legacy browsers, all tabs and UI lived in one single process. A bug in one tab crashed the entire program. Chromium solved this by functioning like an operating system: it separates high-privilege system tasks from untrusted web code across isolated processes connected by **Mojo IPC**.

Browser Process (The Supervisor) FULL OS PRIVILEGES

Owns native OS window handles, disk profile I/O, raw TCP/UDP sockets, and tab lifecycle. If this crashes, the browser terminates.

Renderer Process (The Worker) ZERO OS PRIVILEGES (SANDBOXED)

Runs the Blink layout engine and V8 JS engine. Has zero disk or socket access. Intercepted by Linux Seccomp-BPF filters.

Zygote Process (The Template) LINUX OPTIMIZATION

A warm, pre-initialized process forked on-demand to create new renderers in milliseconds without cold startup lag.

GPU & Network Processes ISOLATED WORKERS

Handles hardware-accelerated compositor rendering (Vulkan/GL) and network TLS/HTTP stack in dedicated sandbox boundaries.

02 The "Shim Pattern": How Obscure Stays Sustainable

The Rebase Trap & The 10-Line Rule

When teams fork Chromium, they typically patch dozens of core files. When upstream releases a milestone update with 50,000 commits, they spend months resolving merge conflicts.

Obscure's Shim Architecture: Over 98% of Obscure code lives inside an isolated module directory: `//chrome/browser/obscure/`. Upstream Chromium is modified in only **two files** totaling under 10 lines:

- **HK-01 (`chrome_web_ui_configs.cc`):** ~3 lines registering `ObscureNewTabPageUIConfig` on `chrome::kChromeUINewTabPageHost`.
- **HK-02 (`chrome/browser/BUILD.gn`):** ~2 lines appending `"//chrome/browser/obscure"` to dependencies.

03 Git Workflow: The 4-Tier Rollback Fortress

Why Hard Resets Are 100% Safe When Tagged

Chromium's git history exceeds 40 million lines. To prevent catastrophic data loss, Obscure structures work into four distinct safety tiers:

TIER 1 **Git Revert (Non-Destructive Inversion):** `git revert <commit-id>` — Creates a new commit that cleanly inverts code changes while preserving full commit history.

TIER 2 **Patch Reversal (Dry-Run Undo):** `git apply --reverse <patch>` — Reverses an applied patch series without altering branch heads.

TIER 3 **Baseline Hard Reset (The Clean Return):** `git reset --hard obscure-baseline-155.0.8059.39` — Restores working tree and index to the immutable baseline tag.

TIER 4 **Git Bundles (Work Recovery Fortress):** `git bundle create backups/obscure.bundle ...` — Packages commits into binary bundles that recover 100% identically even after a hard reset.

Line-Ending Defense: `*.patch -text`

Windows uses CRLF (`\r\n`); Linux uses LF (`\n`). If Windows Git converts line endings in a patch file, `git am` fails in WSL2. Adding `*.patch -text` to `.gitattributes` forces Git to treat all patches as raw binary streams, eliminating corruption.

04 Script Tooling: Understanding What Runs Under the Hood

Script Tool	Mechanics & Flags	What It Physically Proves
`sync_sources.py`	Computes SHA-256 of native files; copies from Windows to WSL2 tree without symlinks.	Prevents cross-filesystem NTFS/ext4 index corruption and Ninja dependency loops.
`test_tb10_patch_revert.sh`	Runs `git apply --check`, applies patch via `git am`, and resets to baseline tag.	Proves TB-10: patches apply cleanly and revert with 100% cryptographic hash identity.
`check_sandbox.py`	Inspects `/proc/<pid>/status` for `Seccomp: 2` and `NoNewPrivs: 1` on live renderers.	Empirically proves the multi-process sandbox jail is active without using `--no-sandbox`.
`demonstrate_rollback_recovery.sh`	Simulates work, creates bundle, resets to baseline, and restores from bundle.	Empirically proves zero work loss: `BEFORE_COMMIT == AFTER_COMMIT`.

05 Founder Oral Defense Cheat Sheet (Mastery Q&A)

Q1: "Why build on Chromium C++ instead of an Electron wrapper?"

Founder Answer: Electron bundles an entire Node.js runtime and separate Chromium instance per application, consuming massive RAM and exposing insecure OS-level APIs directly to web renderers. Obscure modifies Chromium directly at the C++ engine layer, preserving full hardware-accelerated Blink rendering and Linux kernel Seccomp-BPF sandboxing while allowing deep developer tooling integration.

Q2: "How do you guarantee user privacy in Incognito windows?"

Founder Answer: We enforce privacy in C++ at the routing layer. In `ObscureNewTabPageUIConfig::IsWebUIEnabled()`, we check `!profile->IsOffTheRecord()`. If the user opens an Incognito or Guest window, this returns false, causing Chromium to automatically fall back to its native private screens. Obscure custom UI and workspace data never execute inside private sessions.

Q3: "How can you be sure an upstream Chromium update won't break Obscure?"

Founder Answer: We use the Shim Pattern. Over 98% of Obscure code lives in an isolated module directory (`//chrome/browser/obscure/`). Our external touchpoints in upstream Chromium are strictly limited to two registration hooks (HK-01 and HK-02) totaling under 10 lines of diff. Rebasing onto a new Chromium milestone only requires re-verifying those two registration calls.